



- Name : Harshvardhan Patil
- Branch : AI&DS
- Roll No : SY B 23
- Batch : SY B2

AIM: Use sequence data types and their associated operations in Python a) Strings b) Lists c) Arrays d) Tuples e) Dictionary f) Sets g) Range

Question 1: What are sequence data types in Python? Name them and explain how indexing and slicing work across different sequence types. Sequence data types in Python are ordered collections of elements where each element is accessible by its position (index). Python supports the following sequence types: strings, lists, tuples, arrays, and range objects. Dictionaries and sets are not true sequences since they are unordered (or ordered by insertion in modern Python, but not index-accessible the same way). Indexing allows access to individual elements using their position. Python uses zero-based indexing, meaning the first element is at index 0. Negative indexing is also supported, where -1 refers to the last element, -2 to the second last, and so on. This works consistently across strings, lists, tuples, arrays, and ranges. Slicing allows extraction of a portion of a sequence using the syntax `sequence[start:stop:step]`. The start index is inclusive, the stop index is exclusive, and step defines the interval. If start is omitted, it defaults to 0; if stop is omitted, it goes to the end; if step is omitted, it defaults to 1. A step of -1 reverses the sequence. Slicing returns a new object of the same type (except for arrays, which may return a list depending on usage).

Question 2: What is the difference between a list and a tuple in Python? Why would you prefer a tuple over a list in real-world applications? A list is a mutable, ordered sequence that allows adding, removing, or modifying elements after creation. A tuple is an immutable, ordered sequence where elements cannot be changed once assigned. Lists use square brackets and tuples use parentheses. Because tuples are immutable, they consume less memory and are slightly faster to access than lists. In real-world applications, tuples are preferred when data should not be modified, such as storing geographic coordinates, RGB color values, database records returned from queries, or configuration constants. Tuples can also be used as dictionary keys (since they are hashable), whereas lists cannot. When you want to signal to other developers that a collection of values is fixed and should not change, a tuple is the appropriate choice over a list.

Question 3: How are Python arrays different from lists? When should you use the array module instead of a list? Python lists can store elements of different data types in the same collection, making them flexible but memory-inefficient for large

numerical data. The array module provides arrays that store elements of a single, fixed data type (specified using a type code such as 'i' for integer or 'f' for float). This makes arrays more memory-efficient and faster for numerical operations compared to lists. You should use the array module instead of a list when you are working with a large collection of numeric data of a uniform type and memory efficiency matters. For example, when handling binary data, working with hardware buffers, or storing large datasets of integers or floats where you do not need the overhead of a list. However, for advanced numerical computing, NumPy arrays are generally preferred over the built-in array module because NumPy offers far more functionality.

Question 4: Explain how dictionaries work internally in Python. Why are dictionary lookups faster compared to lists or tuples? Internally, Python dictionaries are implemented using a hash table. When a key-value pair is stored, Python computes a hash value for the key using a built-in hash function. This hash value determines the memory location (bucket) where the value is stored. When you look up a key, Python computes the hash of the key again and directly jumps to the corresponding bucket, retrieving the value in constant time on average, denoted as $O(1)$. Lists and tuples, on the other hand, require linear search when you are looking for a specific value, meaning Python must scan elements one by one, which takes $O(n)$ time in the worst case. This is why dictionary lookups are significantly faster than searching through a list or tuple. Keys in a dictionary must be hashable (immutable types like strings, integers, or tuples), which ensures the hash function produces consistent results.

Question 5: What is the difference between sets and lists in Python? How do sets handle duplicate values, and where are sets useful in practical scenarios? A list is an ordered, mutable sequence that allows duplicate elements and supports indexing. A set is an unordered, mutable collection that does not allow duplicate elements and does not support indexing. Sets are implemented using a hash table (similar to dictionaries), which makes membership testing much faster in sets compared to lists. When you add a duplicate value to a set, it is automatically ignored and only one copy is retained. For example, adding 5 to a set that already contains 5 will not change the set. Sets are useful in practical scenarios such as removing duplicates from a dataset, finding common elements between two collections using intersection, identifying unique items in membership checks, and performing mathematical set operations like union, intersection, and difference. They are especially helpful in data cleaning tasks where uniqueness of values matters.

```
In [ ]: s = "Python"
        print(s[0], s[-1])
```

```

print(s[1:4])
print(s[::-2])

l = [10, 20, 30, 40, 50]
print(l[2])
print(l[1:4])
print(l[::-1])

t = (1, 2, 3, 4, 5)
print(t[0])
print(t[2:5])

import array
a = array.array('i', [1, 2, 3, 4, 5])
print(a[1])
print(a[1:3])

r = range(0, 10, 2)
print(r[2])
print(list(r[1:4]))

my_list = [1, 2, 3]
my_list.append(4)
my_list[0] = 99
print(my_list)

my_tuple = (10, 20, 30)
print(my_tuple[1])

coordinates = (19.9975, 73.7898)
print(coordinates)

d = {my_tuple: "location"}
print(d)

import array

general_list = [1, "hello", 3.14, True]
print(general_list)

int_array = array.array('i', [10, 20, 30, 40])
print(int_array)
print(int_array[2])

float_array = array.array('f', [1.1, 2.2, 3.3])
print(float_array)
student = {
    "name": "Harshu",
    "branch": "AIDS",
    "semester": 4
}

```

```
print(student["name"])
print(student.get("semester"))

student["cgpa"] = 9.1
print(student)

print(hash("name"))
print(hash(42))
my_list = [1, 2, 2, 3, 4, 4, 5]
print(my_list)

my_set = {1, 2, 2, 3, 4, 4, 5}
print(my_set)

my_set.add(5)
print(my_set)

a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a & b)
print(a | b)
print(a - b)

items = ["apple", "banana", "apple", "mango", "banana"]
unique_items = set(items)
print(unique_items)

print(3 in my_set)
print(3 in my_list)
```

```

P n
yth
Pto
30
[20, 30, 40]
[50, 40, 30, 20, 10]
1
(3, 4, 5)
2
array('i', [2, 3])
4
[2, 4, 6]
[99, 2, 3, 4]
20
(19.9975, 73.7898)
{(10, 20, 30): 'location'}
[1, 'hello', 3.14, True]
array('i', [10, 20, 30, 40])
30
array('f', [1.100000023841858, 2.2000000047683716, 3.299999952316284])
Harshu
4
{'name': 'Harshu', 'branch': 'AIDS', 'semester': 4, 'cgpa': 9.1}
-4031297978559787675
42
[1, 2, 2, 3, 4, 4, 5]
{1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
{3, 4}
{1, 2, 3, 4, 5, 6}
{1, 2}
{'apple', 'banana', 'mango'}
True
True

```

Conclusion – Python provides a rich set of sequence and collection data types, each designed for a specific purpose. Strings, lists, tuples, arrays, and ranges support indexing and slicing, making data access straightforward and consistent. Lists offer flexibility through mutability, while tuples provide safety and efficiency through immutability, making each suitable for different scenarios. The array module fills the gap when uniform numeric data needs to be stored efficiently without the overhead of a general-purpose list. Dictionaries offer blazing-fast lookups through hash-based internal storage, making them ideal for key-value mapping in real applications. Sets eliminate duplicates automatically and support powerful mathematical operations, making them invaluable in data cleaning and membership testing tasks. Understanding when and why to use each data type is a fundamental skill in Python programming, directly impacting the performance, readability, and correctness of any data science or software project.